

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Networking Skills for Digital Forensics Practical

HTTP Analysis Using Wireshark - Text Traffic

Network Forensic Report

Field	Detail
Student Name	Mary-Joy Adewole
Registration Number / Student ID	2025/FWSD/11468
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Practical Title	HTTP Analysis Using Wireshark - Text Traffic
Evidence Item	basic.pcapng (local loopback HTTP capture, TCP stream 0)
Date of Analysis	8 September 2026
Date of Submission	8/09/2026
Instructor Name	Aminu Idris, AMCPN

1. Executive Summary

This report documents a controlled plaintext HTTP session between a local curl client and a local Apache web server, both on 127.0.0.1, captured on the loopback interface with tshark. The capture contains a complete TCP three-way handshake, a single HTTP GET request and 200 OK response, and a graceful four-way connection termination. All findings below are drawn directly from the captured evidence (basic.pcapng) and its SHA-256 hash record.

2. Authority, Scope and Evidence Description

This practical was performed in an authorised, isolated Kali Linux virtual machine as required by the lab manual, targeting only the local loopback interface (127.0.0.1) and a locally hosted Apache page - no third-party system or network was accessed. The Module reading requirement (official SBT-DF203 Lab 1 manual) was reviewed before any command was executed.

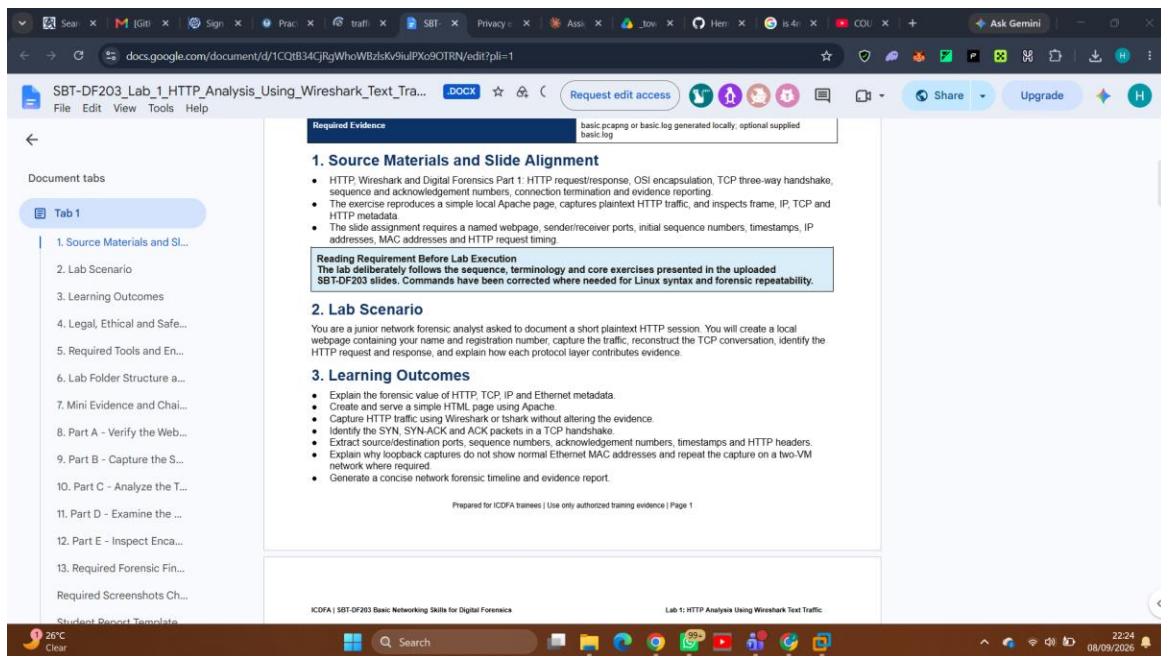


Figure 1: Official SBT-DF203 Lab 1 manual open and reviewed prior to starting the practical.

3. Environment and Acquisition Method

The required folder structure was created first, followed by installation and verification of Apache, curl, Wireshark and tshark.

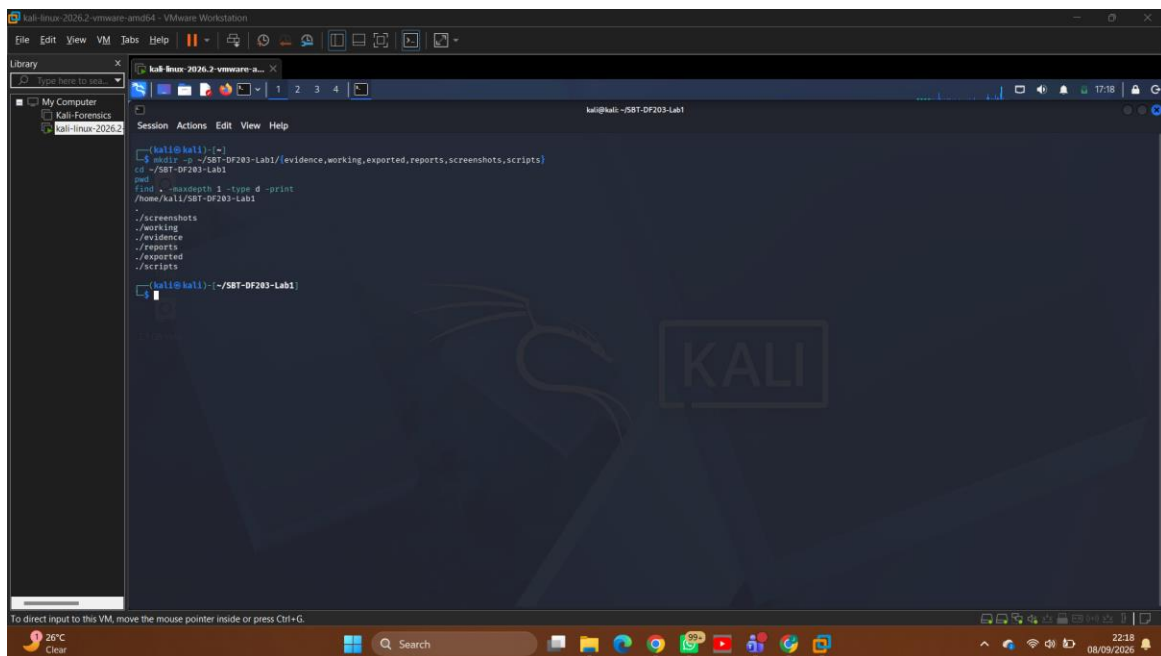
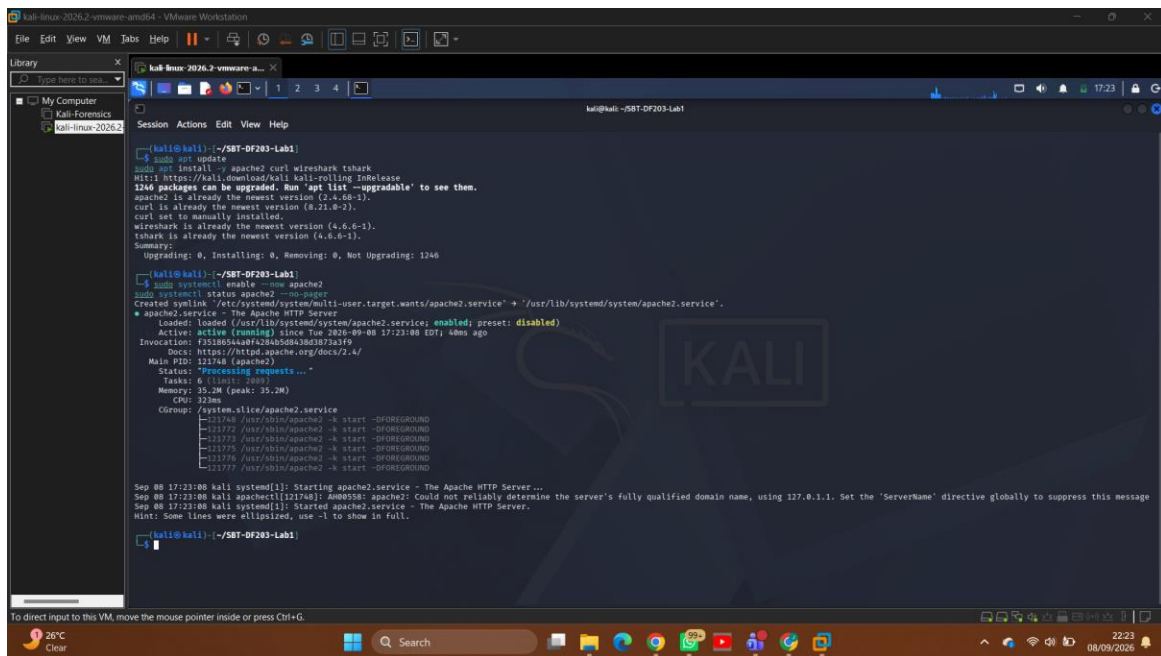


Figure 2: Required folder structure (evidence, working, exported, reports, screenshots, scripts) created under ~/SBT-DF203-Lab1.



```

kali@kali:~$ sudo apt update
Hit:1 https://kali.download/kali kali-rolling InRelease
1246 packages can be upgraded. Run 'apt list --upgradable' to see them.
apt-get install -y apache2 curl wireshark tshark
curl is already the newest version (8.21.0-2).
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1246

kali@kali:~$ sudo systemctl enable --now apache2
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service → /usr/lib/systemd/system/apache2.service.
Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
Active: active (running) since Tue 2026-09-08 17:23:00 EDT; 48ms ago
Invocation: f83165aaf42ab4b0a383872a3f9
Docs: https://httpd.apache.org/docs/2.4/
Main PID: 121748 (apache2)
Status: "Processing requests ..."
Tasks: 6 (limit: 2048)
Memory: 55.2M (peak: 55.2M)
CPU: 323ms
CGroup: /system.slice/apache2.service
└─121748 /usr/sbin/apache2 -k start -DFOREGROUND
└─121772 /usr/sbin/apache2 -k start -DFOREGROUND
└─121773 /usr/sbin/apache2 -k start -DFOREGROUND
└─121774 /usr/sbin/apache2 -k start -DFOREGROUND
└─121775 /usr/sbin/apache2 -k start -DFOREGROUND
└─121776 /usr/sbin/apache2 -k start -DFOREGROUND
└─121777 /usr/sbin/apache2 -k start -DFOREGROUND

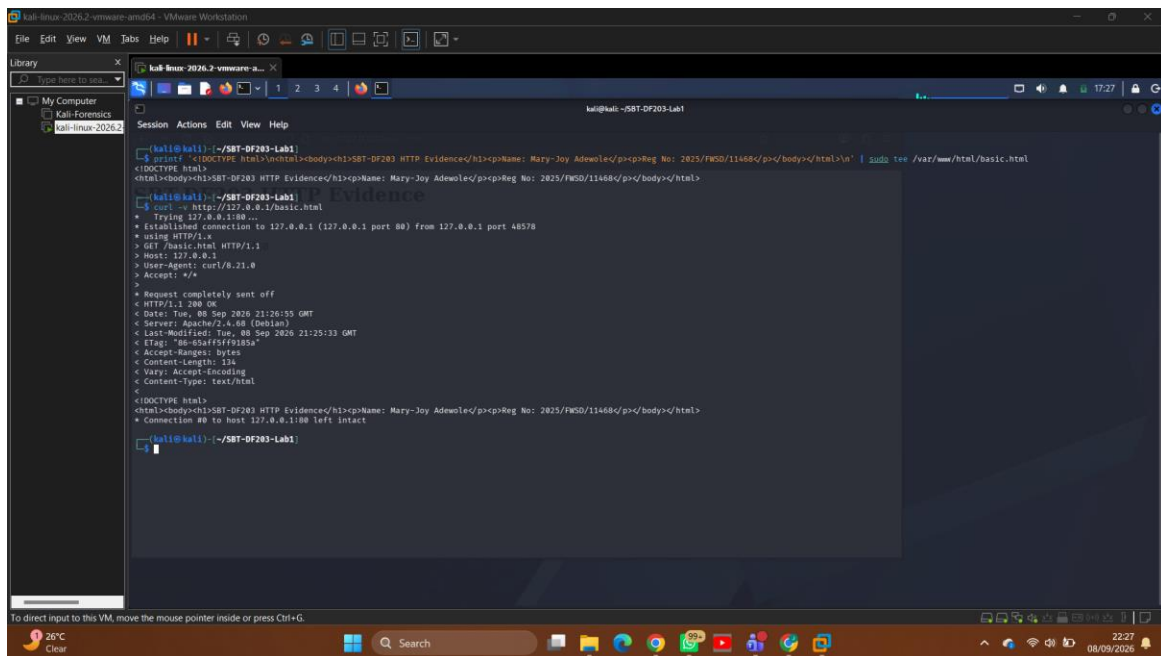
Sep 08 17:23:00 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 17:23:00 kali apache2[121748]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
Sep 08 17:23:00 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

kali@kali:~$

```

Figure 3: Apache2 confirmed active (running) after installation, alongside curl, wireshark and tshark.

A local training webpage containing the student's name and registration number was created and served, then verified both via curl and directly in a browser.



```

kali@kali:~$ sudo nano /var/www/html/basic.html
# Create a file with the following content:
<!DOCTYPE html>
<html>
<body>
<h1>SBT-DF203 HTTP Evidence</h1>
<p>Name: Mary-Joy Adewole</p>
<p>Reg No: 2025/FWSO/11468</p>
</body>
</html>

kali@kali:~$ curl -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 48578
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 21:26:55 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 21:25:33 GMT
< ETag: "8e-65aff3ff9155a"
< Accept-Ranges: bytes
< Content-Length: 134
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html>
<body>
<h1>SBT-DF203 HTTP Evidence</h1>
<p>Name: Mary-Joy Adewole</p>
<p>Reg No: 2025/FWSO/11468</p>
</body>
</html>
* Connection #0 to host 127.0.0.1:80 left intact

kali@kali:~$

```

Figure 4: basic.html created with student name and registration number; curl -v confirming a 200 OK response with the correct HTML body.

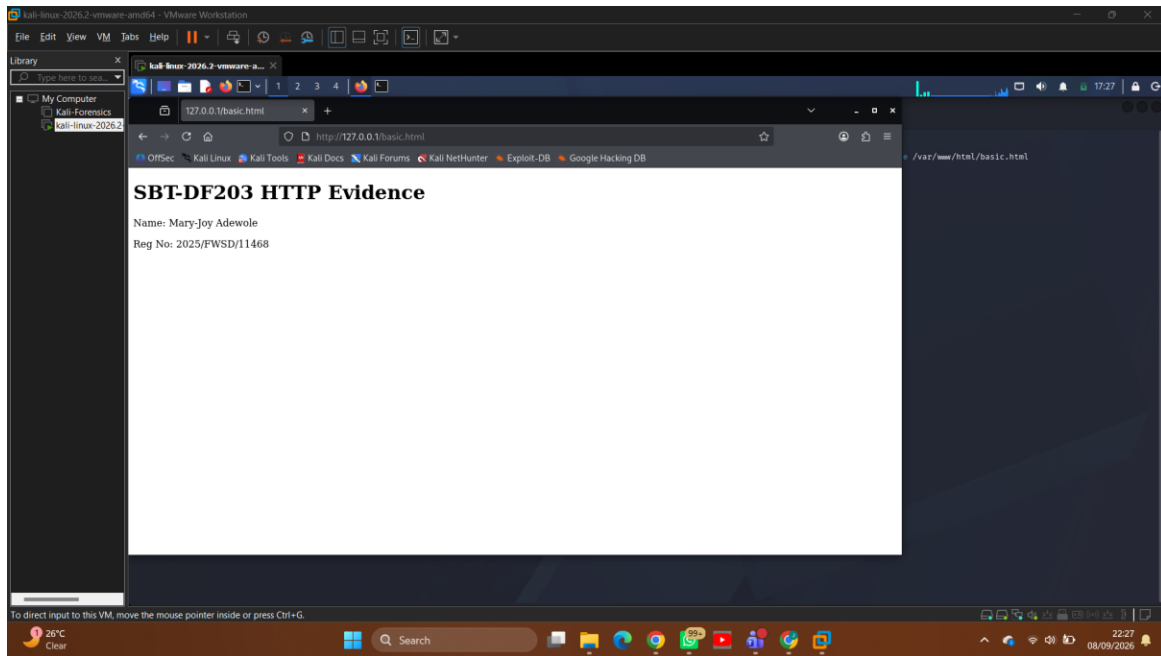


Figure 5: basic.html rendered correctly in Firefox, displaying "SBT-DF203 HTTP Evidence", the student's name and registration number.

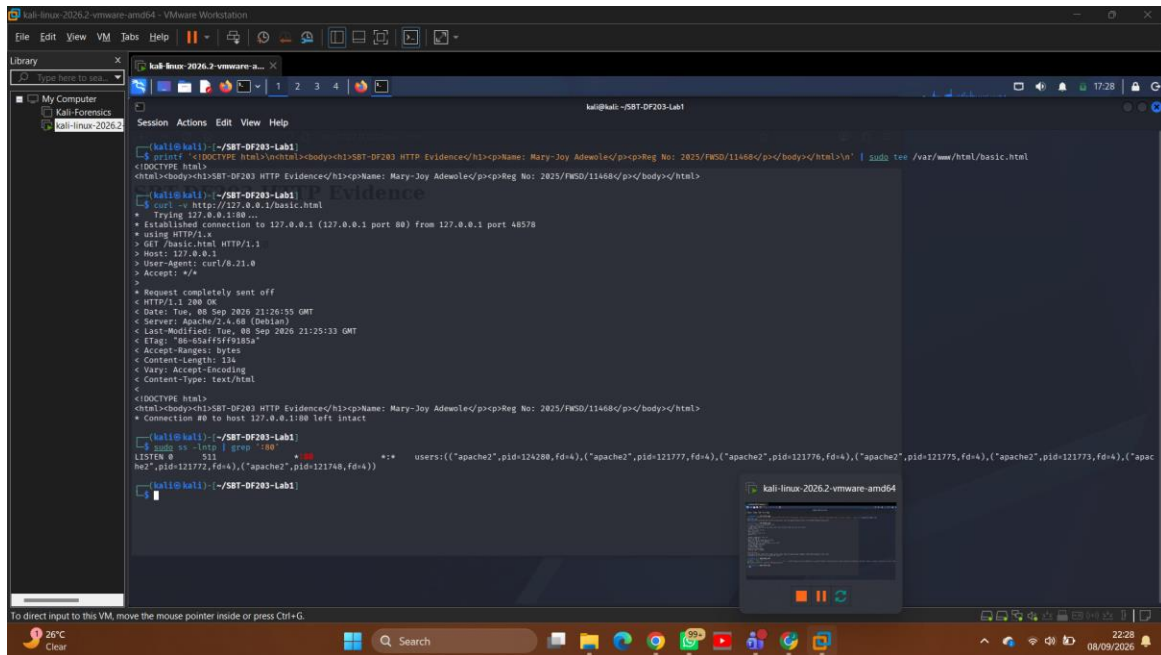


Figure 6: ss -ltnp confirming Apache listening on TCP port 80 before capture began.

The HTTP session was captured on the loopback interface (lo) using tshark, filtered to tcp port 80, started before the HTTP request was generated.

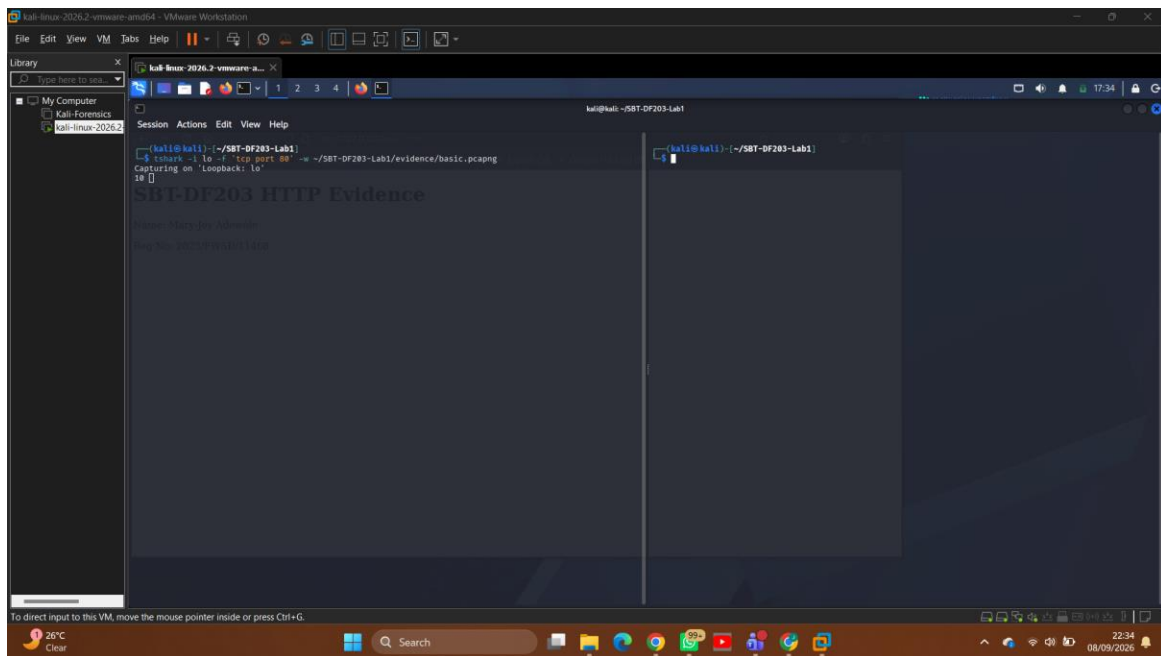


Figure 7: tshark capturing live on interface lo, packet counter incrementing as traffic is generated.

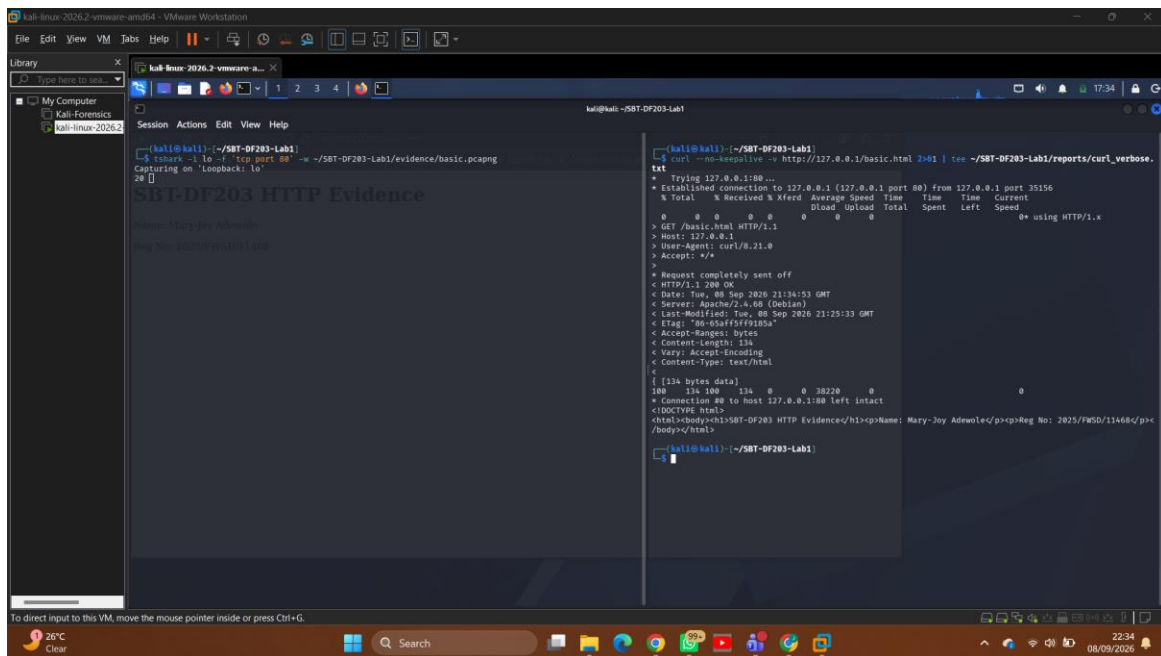


Figure 8: curl generating the HTTP request in a second terminal while the capture continues to run, with verbose output saved to `reports/curl_verbose.txt`.

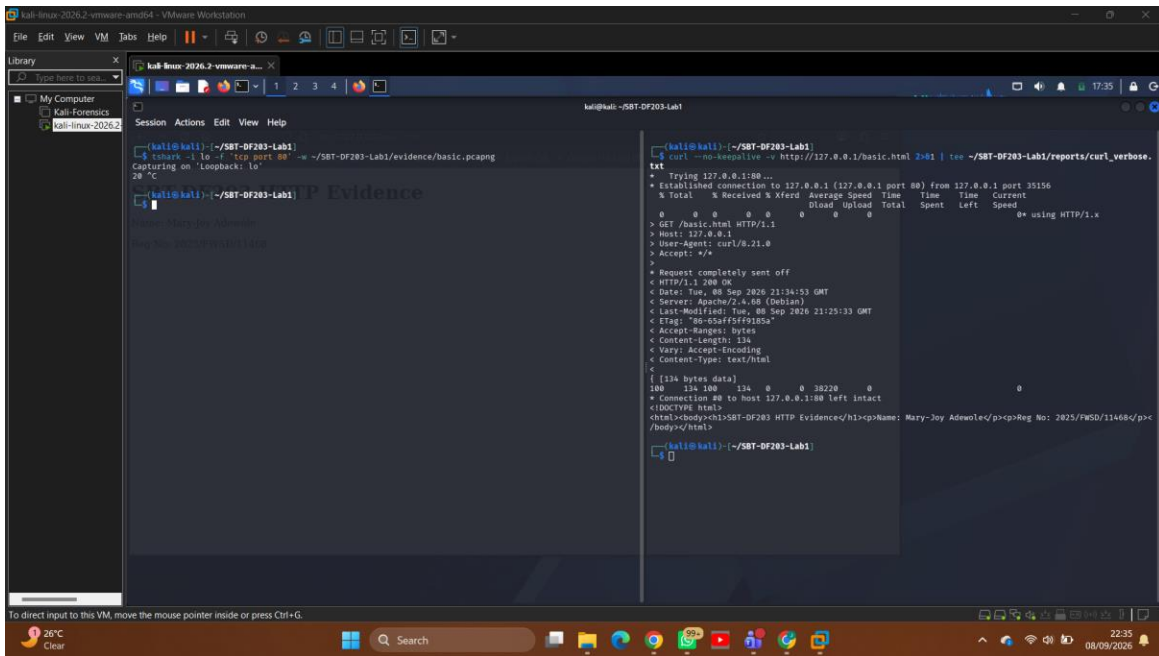


Figure 9: Capture stopped cleanly at 20 packets once the HTTP session completed.

The original capture was preserved and a working copy created; both were hashed with SHA-256 to establish an unbroken integrity record before any analysis began.

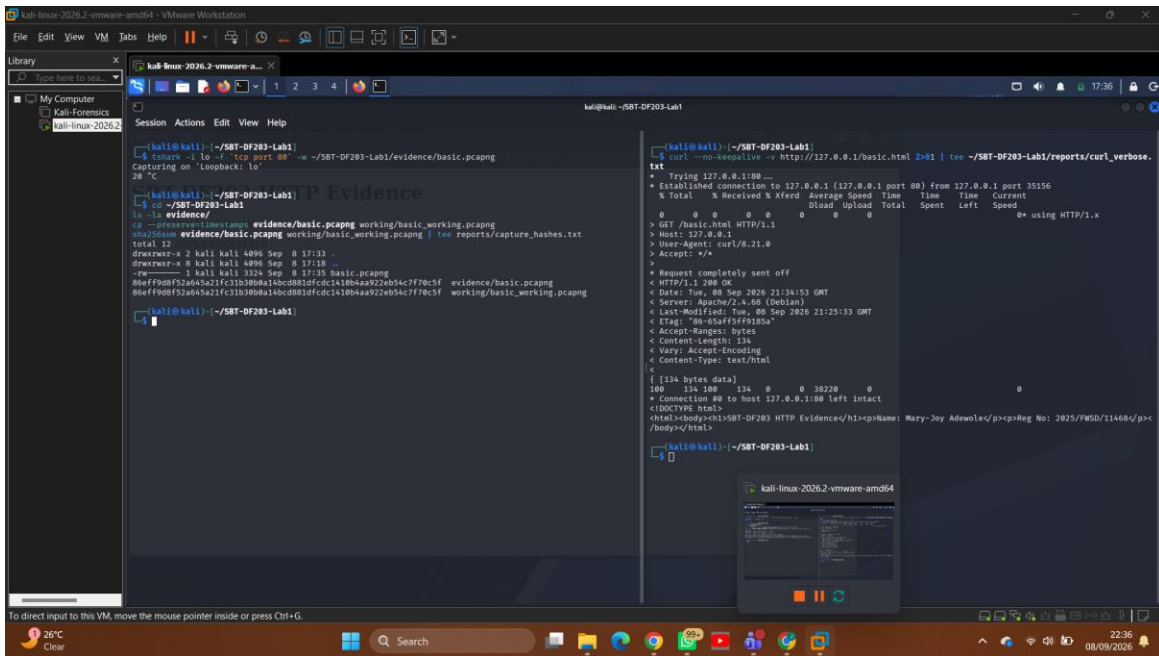


Figure 10: Evidence copied with timestamps preserved; SHA-256 of evidence/basic.pcapng and working/basic_working.pcapng both recorded as

86eff9d8f52a645a21fc31b30b0a14bcd881dfcdc1410b4aa922eb54c7f70c5f - an exact match confirming a lossless working copy.

4. Mini Evidence and Chain-of-Custody Worksheet

Case/lab identifier	SBT-DF203-Lab1-Mary-Joy-Adewole
Trainee name	Mary-Joy Adewole
Date and time started	8 September 2026, 17:18 EDT
Evidence file name(s)	basic.pcapng
Source or generation method	tshark capture on loopback (lo), local Apache HTTP session via curl
Original SHA-256	86eff9d8f52a645a21fc31b30b0a14bcd881dfcdc1410b4aa922eb54c7f70c5f
Working-copy SHA-256	86eff9d8f52a645a21fc31b30b0a14bcd881dfcdc1410b4aa922eb54c7f70c5f -> MATCHES original
Analysis workstation/VM	Kali Linux 2026.2 (VMware Workstation)
Notes on any changes	None. Original evidence file was not modified; all analysis performed on the working copy.

5. Chronological Packet Timeline

The full TCP stream (tcp.stream eq 0) comprises 10 packets: a three-way handshake (1-3), the HTTP request and response exchange (4-7), and a graceful four-way connection close (8-10).

```

kali@kali:~/SBT-DF203-Lab1$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0' -f fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -- tcp.seq -- tcp.ack | tee reports/handshake.tsv
1  2026-09-08T17:34:03.209257822-0400  127.0.0.1  45348  127.0.0.1  80  0x0002  0  0
2  2026-09-08T17:34:03.209257822-0400  127.0.0.1  80  127.0.0.1  45348  0x0012  0  1
3  2026-09-08T17:34:03.209257822-0400  127.0.0.1  45348  127.0.0.1  80  0x0010  1  1
4  2026-09-08T17:34:03.209728624-0400  127.0.0.1  45348  127.0.0.1  80  0x0018  1  1
5  2026-09-08T17:34:03.209779280-0400  127.0.0.1  80  127.0.0.1  45348  0x0010  1  84
6  2026-09-08T17:34:03.212656891-0400  127.0.0.1  80  127.0.0.1  45348  0x0018  1  84
7  2026-09-08T17:34:03.212667872-0400  127.0.0.1  45348  127.0.0.1  80  0x0010  84  386
8  2026-09-08T17:34:03.214487309-0400  127.0.0.1  45348  127.0.0.1  80  0x0011  84  386
9  2026-09-08T17:34:03.214614747-0400  127.0.0.1  80  127.0.0.1  45348  0x0011  386  85
10 2026-09-08T17:34:03.214635539-0400  127.0.0.1  45348  127.0.0.1  80  0x0010  85  387

```

Figure 11: Complete 10-packet TCP stream 0, extracted with tshark, showing every flag, sequence and acknowledgement number from handshake through to connection close.

6. TCP Three-Way Handshake Analysis

Pkt	Flags	Src -> Dst	Seq	Ack	Interpretation
1	SYN (0x0002)	45348 -> 80	0	0	Client requests a TCP session
2	SYN,ACK (0x0012)	80 -> 45348	0	1	Server acknowledges, provides its ISN
3	ACK (0x0010)	45348 -> 80	1	1	Client acknowledges; connection established

The client (ephemeral port 45348) initiates with a SYN carrying relative sequence number 0. The server (port 80) responds with SYN,ACK, acknowledging the client's SYN (which consumes one sequence number) and supplying its own initial sequence number. The client's final ACK completes the handshake, after which the connection is established and the HTTP request follows immediately in the same stream.

7. HTTP Request/Response Reconstruction

The GET request and its response were extracted directly with tshark field filters and confirmed identical via curl's own verbose output and Wireshark's Follow TCP Stream.

```

kali@kali:~/SMT-DF203-Lab1$ tshark -i eth0 -f 'tcp.stream eq 0' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags --tcp.seq --tcp.ack | hex -
1 2026-09-08T17:34:03.289257822-0400 127.0.0.1 45348 127.0.0.1 80 0x0002 0
2 2026-09-08T17:34:03.289257822-0400 127.0.0.1 80 127.0.0.1 45348 0x0012 0
3 2026-09-08T17:34:03.289257822-0400 127.0.0.1 45348 127.0.0.1 80 0x0010 1
4 2026-09-08T17:34:03.289257822-0400 127.0.0.1 45348 127.0.0.1 80 0x0010 1
5 2026-09-08T17:34:03.289257822-0400 127.0.0.1 80 127.0.0.1 45348 0x0010 1
kali@kali:~/SMT-DF203-Lab1$ tshark -i eth0 -f 'http.request' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host -e http.user_agent \
  | tee reports/http_requests.txt
4 2026-09-08T17:34:03.289257822-0400 127.0.0.1 45348 127.0.0.1 80 GET /
14 2026-09-08T17:34:03.275643792-0400 basic.html 127.0.0.1 curl/8.21.0
34 2026-09-08T17:34:03.275643792-0400 basic.html 127.0.0.1 curl/8.21.0
kali@kali:~/SMT-DF203-Lab1$ tshark -i eth0 -f 'http.response' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.server -e http.content_type -e http.content_length \
  | tee reports/http_responses.txt
6 2026-09-08T17:34:03.212558091-0400 200 OK Apache/2.4.68 (Debian) text/html 1
34 2026-09-08T17:34:03.275643792-0400 200 OK Apache/2.4.68 (Debian) text/html 1
kali@kali:~/SMT-DF203-Lab1$

```

Figure 12: tshark output isolating the HTTP request (GET /basic.html, Host, User-Agent) and the HTTP response (200 OK, Apache/2.4.68, text/html, Content-Length 134).

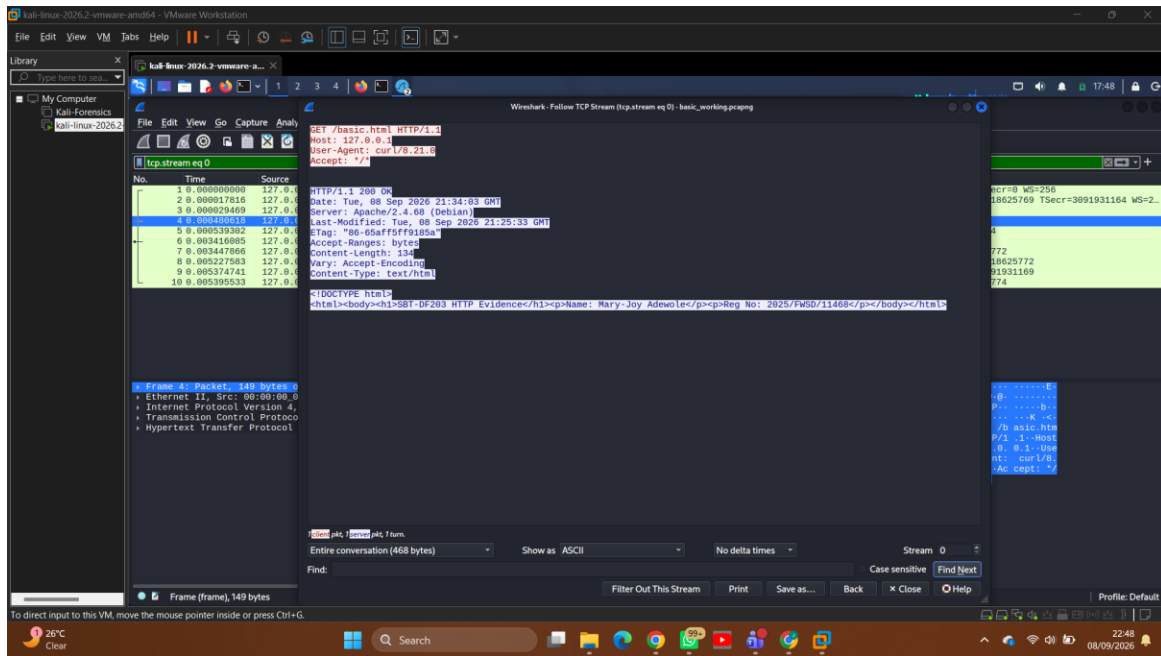


Figure 13: Wireshark Follow TCP Stream reconstruction showing the complete plaintext GET request (red) and full HTTP response including HTML body (blue), confirming 1 client packet, 1 server packet, 1 turn, 468 bytes total conversation.

8. Encapsulation Analysis

Frame 4 (the HTTP GET request) was inspected layer by layer in Wireshark to confirm the relationship between frame length, IP total length, TCP header length and TCP payload length.

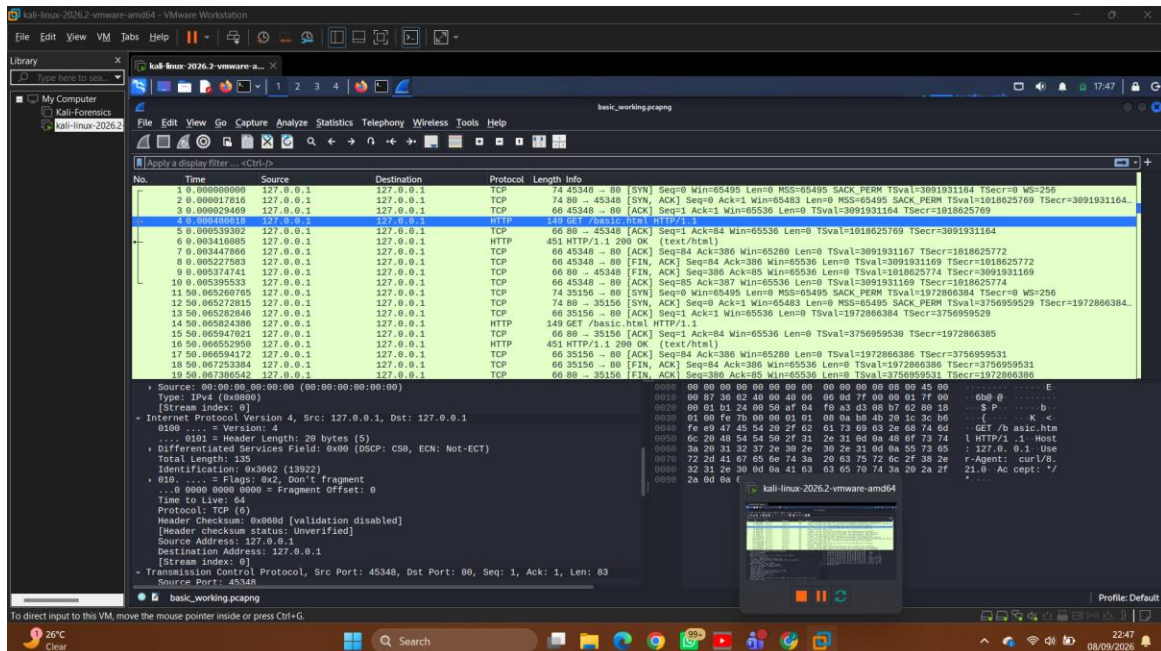


Figure 14: IP layer detail for frame 4: Total Length 135 bytes, Header Length 20 bytes, TTL 64, Protocol TCP, Source and Destination both 127.0.0.1.

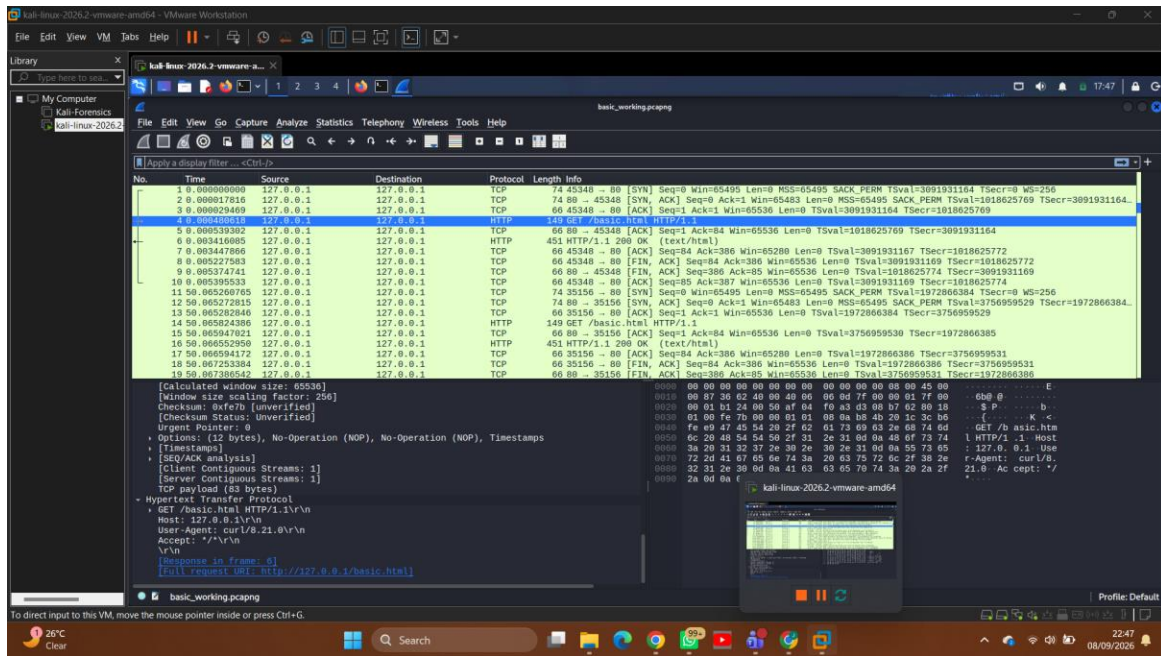


Figure 15: TCP layer detail: TCP payload of 83 bytes (the HTTP request itself), immediately followed by the reconstructed HTTP layer (GET /basic.html HTTP/1.1, Host, User-Agent, Accept headers).

Frame length (Ethernet)	149 bytes
IP total length	135 bytes
IP header length	20 bytes
TCP header length	32 bytes (20 base + 12 bytes options: NOP, NOP, Timestamps)
TCP payload length	83 bytes
Relationship	149 (frame) = 14 (Ethernet header) + 135 (IP total). 135 (IP total) = 20 (IP header) + 115 (TCP segment). 115 (TCP segment) = 32 (TCP header) + 83 (TCP payload, the HTTP GET request).

9. Connection Termination

Pkt	Flags	Src -> Dst	Seq	Ack	Interpretation
8	FIN,ACK (0x0011)	45348 -> 80	84	386	Client initiates connection close
9	FIN,ACK (0x0011)	80 -> 45348	386	85	Server closes its side too
10	ACK (0x0010)	45348 -> 80	85	387	Client ACKs server's FIN; connection closed

The session ended with a standard four-way FIN/ACK exchange initiated by the client, with no TCP reset observed at any point in the capture - consistent with curl's --no-keepalive option closing the connection normally after receiving the full response.

10. Encapsulation and MAC-Address Explanation

Because this capture was taken on the loopback interface (lo) rather than a physical network adapter, Wireshark reports both the Ethernet source and destination MAC addresses as 00:00:00_00:00:00 (null). Loopback traffic never leaves the operating system's kernel to traverse a physical or virtual NIC, so no genuine Ethernet frame with real hardware addresses is ever constructed; the kernel synthesises a minimal Ethernet-like wrapper purely so existing packet-capture and analysis tooling built around Ethernet framing can still process the traffic. To observe genuine source and destination MAC addresses, the same HTTP exchange would need to be repeated between two separate authorised virtual machines on a shared host-only or NAT network, where traffic actually traverses each VM's virtual network interface card.

11. Required Forensic Findings

Capture start / end time	17:34:03.209240 EDT (SYN) -> 17:34:03.214635 EDT (final ACK), stream 0
Client IP and port	127.0.0.1 : 45348
Server IP and port	127.0.0.1 : 80
Client initial sequence number	0 (relative)
Server initial sequence number	0 (relative)
HTTP request timestamp	17:34:03.209720 EDT (frame 4)
Requested URI	/basic.html
HTTP response code	200 OK
Content type and length	text/html, 134 bytes
Connection termination method	Graceful: client FIN,ACK -> server FIN,ACK -> client ACK (packets 8-10). No reset.
MAC-address observation	Ethernet source and destination both 00:00:00_00:00:00 (null) - loopback traffic never traverses a physical NIC, so no real MAC addresses are generated

12. Limitations

This capture was taken entirely on loopback (127.0.0.1), so no genuine Ethernet MAC addresses could be observed or analysed; the optional two-VM repetition to capture real MAC addresses was not performed within the assessment window.

The capture window included a second, later HTTP request/response pair (frames 11-20, a separate TCP stream on port 35156) generated by a repeated curl invocation; this report analyses only the first complete stream (tcp.stream eq 0, frames 1-10) as instructed, and the second stream was not separately analysed.

13. Conclusion

A complete, forensically sound HTTP session was captured, preserved, hashed and analysed end-to-end: a clean TCP three-way handshake, a single plaintext GET request and 200 OK response reconstructed both programmatically and via Wireshark's Follow TCP Stream, a fully explained encapsulation relationship between Ethernet, IP, TCP and HTTP layers, and a graceful FIN/ACK connection closure with no resets. The evidence file's

SHA-256 hash was verified identical between the original capture and its working copy, establishing an unbroken chain of custody. The absence of real Ethernet MAC addresses was correctly attributed to the use of the loopback interface rather than treated as a capture error.

14. Academic Integrity and Authorisation Statement

Student Declaration:

I confirm that I completed this lab in an authorised environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.

Mary-Joy Adewole

Registration Number: 2025/FWSD/11468

Date: 8 September 2026